Refer to https://www.w3schools.com/cpp/cpp_getstarted.asp for syntax

Only use the following includes: iostream, string

Do not use any container from STL

Find the following files inside the assignment folder:

point.h, point.cpp, and main.cpp

Class Point is ***declared in point.h*** and ***defined in point.cpp***

main.cpp has the driver program that takes user commands, creates objects of type Point accordingly and performs actions on/for it.

Point class represents a 2D point (x and y components) and has one 'alias'.

Refer to the .h file for a detailed description of the class.

Lab Submittable Tasks:

Files to be submitted: point.h, point.cpp

0. **Read the code comments in point.h to understand the class**
1. Complete Point class declaration in point.h, by including the following operators for overloading:
    a. = // copy assingment: e.g. p1 = p2 should be doable for two points p1, p2
    b. + // + should add respective x and y components
    c. - // - should subtract respective x and y components
    d. + with a double number // + should add only x component
    e. - with a double number // - should subtract only x component
    f. ++ // pre increment both x and y components
    g. -- // pre decrement both x and y components
    h. == // equality comparison operator
2. Fix Point class definition *syntactically* in point.cpp, so that it compiles using **g++ -c** command without error.
3. ***Create one main() function in the point.cpp***, and implement the following test cases of point operations in the main function itself.

4. Testcases: The following sequence of input actions should match the outputs in sequence.

Input Action	Output from program
Create First Point object p1, with x = 10.1, y = 10.01, alias as “p1”	N/A
Create Second Point object p2, with x = 20.1, y = 20.2, alias as “p2”	N/A
Call p1.show()	Show [p1]: X: 10.1,Y: 10.01
Call p2.show()	Show [p2]: X: 20.1,Y: 20.2
Do p1 + p2	
Call p1.show()	Show [p1-add-p2]: X: 30.2,Y: 30.21
Call p2.show()	Show [p2]: X: 20.1,Y: 20.2
Do p1 - p2	
Call p1.show()	Show [p1-add-p2-diff-p2]: X: 10.1,Y: 10.01
Call p2.show()	Show [p2]: X: 20.1,Y: 20.2
Do p2 + 5.5	
Call p2.show()	Show [p2]: X: 25.6,Y: 20.2
Do p2 – 3.0	
Call p2.show()	Show [p2]: X: 22.6,Y: 20.2
Do --p1	
Call p1.show()	Show [p1-add-p2-diff-p2]: X: 9.1,Y: 9.01
Do ++p2	
Call p2.show()	Show [p2]: X: 23.6,Y: 21.2

In main.cpp, define *Action* **enums (**C++ Enumeration (enum)**)** with the following:

```
CREATE_POINT,
DEEP_COPY_POINT,
ADD_POINT,
DIFF_POINT,
ADD_X_TO_POINT,
ADD_Y_TO_POINT,
COPY_ASSIGN_POINT,
TOGGLE_SHOW_THIS,
GET_POINT_COUNT,
SET_POINT_X,
SET_POINT_Y,
SET_POINT_ALIAS,
DO_PRE_INC,
DO_PRE_DEC,
DO_EXIT
```

a. The function **printMenu()** in this file prints the following menu, and asks the user to enter one choice.
b. Once the user has entered the choice, the program **cout**s the choice, (and then proceeds to further actions), as given below. Any conversion (if needed) between integer and enum class should be using static_cast (static_cast in C++ - GeeksforGeeks).

```
===== Point Actions Menu =====
0. CREATE POINT
1. DEEP COPY POINT
2. ADD POINT
3. DIFF POINT
4. ADD X TO POINT
5. ADD Y TO POINT
6. COPY ASSIGN POINT
7. TOGGLE SHOW THIS
8. GET POINT COUNT
9. SET POINT X
10. SET POINT Y
11. SET POINT ALIAS
12. DO PRE INC
13. DO PRE DEC
14. EXIT
==============================
Enter choice:11
 SET POINT ALIAS
```

c. In the function **runEventLoop(),** that is basically an infinite loop until the user enters choice 14 (EXIT command). In one iteration, it shows all the points created so far, before printing the menu. Here is a sample:

```
========== POINTS REPO =============
Show [point1]: X: 12,Y: 23
Show [point2]: X: 34,Y: 45
========== POINTS REPO END =============
===== Point Actions Menu =====
0. CREATE POINT
1. DEEP COPY POINT
2. ADD POINT
3. DIFF POINT
4. ADD X TO POINT
5. ADD Y TO POINT
6. COPY ASSIGN POINT
7. TOGGLE SHOW THIS
8. GET POINT COUNT
9. SET POINT X
10. SET POINT Y
```

```
11. SET POINT ALIAS
12. DO PRE INC
13. DO PRE DEC
14. EXIT
============================
Enter choice:
```

d. Fill-in the function **doAction()** in this file**,** that takes an enum as parameter, returns void, and has switch-case based on the parameter enum. The code comments explain the Input/Output (cin/cout) behavior of each of the cases. Fill-in all the missing statements.

Take-Home Q2

Read the book.cpp file to understand the requirements.

The book.cpp file given in the assignment defines a book class inside a namespace encyclopedia. You should split it into book.h, and book.cpp, to cover the class declaration and class definition respectively.

Follow the similar design of Point class as above, create book.h and book.cpp.

The following methods need to be there in Book class to provide the listed functionalities in the menu:

1. Constructor
2. Show() // to show book details
3. Overloaded operator + // to add two books
4. Overloaded operator == // to compare two books
5. You should be able to delete all the books objects at once

Create main.cpp for the driver program, move the driver logic from existing book.cpp into main.cpp. Here is a trace of execution, that should be reproducible from your program:

```
========== BOOKS REPO ============
========== BOOKS REPO END =============
===== Book Actions Menu =====
0. CREATE_BOOK
1. SHOW_BOOK
2. ADD_BOOKS
3. DELETE ALL BOOKS
```

```
4. EXIT
==============================
Enter choice: 1
SHOW_BOOK
Enter title: ahahan
ahahan does not exists!
========== BOOKS REPO ============
========== BOOKS REPO END =============
===== Book Actions Menu =====
0. CREATE_BOOK
1. SHOW_BOOK
2. ADD_BOOKS
3. DELETE ALL BOOKS
4. EXIT
==============================
Enter choice: 0
CREATE_BOOK
Enter title: lorem_lipsum
Enter pages: 34
========== BOOKS REPO ============
lorem_lipsum 34
========== BOOKS REPO END =============
===== Book Actions Menu =====
0. CREATE_BOOK
1. SHOW_BOOK
2. ADD_BOOKS
3. DELETE ALL BOOKS
4. EXIT
==============================
Enter choice: 1
SHOW_BOOK
Enter title: lorem_lipsum
lorem_lipsum 34
========== BOOKS REPO ============
lorem_lipsum 34
========== BOOKS REPO END =============
===== Book Actions Menu =====
0. CREATE_BOOK
1. SHOW_BOOK
2. ADD_BOOKS
3. DELETE ALL BOOKS
4. EXIT
==============================
Enter choice: 0
CREATE_BOOK
Enter title: apurbo_ray
Enter pages: 23
========== BOOKS REPO ============
```

```
lorem_lipsum 34
apurbo_ray 23
========== BOOKS REPO END =============
===== Book Actions Menu =====
0. CREATE_BOOK
1. SHOW_BOOK
2. ADD_BOOKS
3. DELETE ALL BOOKS
4. EXIT
==============================
Enter choice: 2
ADD_BOOKS
Enter title 1: lorem_lipsum
Enter title 2: apurbo_ray
========== BOOKS REPO ============
lorem_lipsumapurbo_ray 57
apurbo_ray 23
========== BOOKS REPO END =============
===== Book Actions Menu =====
0. CREATE_BOOK
1. SHOW_BOOK
2. ADD_BOOKS
3. DELETE ALL BOOKS
4. EXIT
==============================
Enter choice: 3
DELETE ALL BOOKS
========== BOOKS REPO ============
========== BOOKS REPO END =============
===== Book Actions Menu =====
0. CREATE_BOOK
1. SHOW_BOOK
2. ADD_BOOKS
3. DELETE ALL BOOKS
4. EXIT
==============================
Enter choice: 0
CREATE_BOOK
Enter title: handbook
Enter pages: 23
========== BOOKS REPO ============
handbook 23
========== BOOKS REPO END =============
===== Book Actions Menu =====
0. CREATE_BOOK
1. SHOW_BOOK
2. ADD_BOOKS
3. DELETE ALL BOOKS
```

```
4. EXIT
==============================
Enter choice: 4
EXIT
Exit
```

For any choice, if you enter a non-existing book name, it should say the book does not exist, i.e. validity checking needs to be present before performing a book operation.